



คำอธิบายการทำงานของระบบ

DevPool 2026

Full-Stack Web Application ด้วย HTML + CSS + JavaScript + Node.js
+ Express



สารบัญ

1. ภาพรวมระบบ
2. package.json - การจัดการ Dependencies
3. server.js - Backend API Server
 1. การตั้งค่า Middleware
 2. In-Memory Database
 3. ฟังก์ชัน Parse ราคาน้ำมัน
 4. API Endpoints ทั้งหมด
4. HTML Frontend - การเชื่อมต่อกับ Backend
5. การทำงานแบบ End-to-End
6. การดีบักและการทดสอบ
7. สถาปัตยกรรมระบบ
8. สรุปและข้อควรระวัง

1. ภาพรวมระบบ

ระบบนี้เป็นเว็บแอปพลิเคชัน Full-Stack ที่ประกอบด้วย:

Frontend (HTML/CSS/JS)

- แสดงโปรไฟล์ผู้ใช้
- ฟอรัมพิมพ์ข้อความแบบ Real-time
- ปุ่ม Like/Unlike พร้อมแสดงจำนวนไลค์
- แสดงราคาน้ำมันบางจาก (iframe)
- ระบบเปลี่ยนธีม (Light/Dark/Auto) และภาษา (ไทย/อังกฤษ)

Backend (Express.js)

- API Endpoints 10+ Route
- In-Memory Database (Likes, Messages)
- Cache System (Oil Price, Users)
- CORS Middleware
- Static File Server

External APIs

- Bangchak Oil Price: <https://oil-price.bangchak.co.th/BcpOilPrice1/th>
- JSONPlaceholder: <https://jsonplaceholder.typicode.com>

2. package.json - การจัดการ Dependencies

```
{
  "name": "devpool-project",
  "version": "1.0.0",
  "description": "DevPool 2026 Project - Full Stack Application",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "node-fetch": "^3.3.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Dependencies ที่สำคัญ

ชื่อ	เวอร์ชัน	คำอธิบาย
express	^4.18.2	Web Framework หลัก ใช้สร้าง API Server, Middleware, Routing
node-fetch	^3.3.2	Library สำหรับทำ HTTP Request ไปยัง API ภายนอก
nodemon	^3.0.1	Dev Dependency - เครื่องมือ reload อัตโนมัติ เมื่อแก้ไขไฟล์

 **หมายเหตุ: สัญลักษณ์** `^` หน้าเวอร์ชันหมายถึงยอมรับเวอร์ชัน patch และ minor ที่สูงกว่า

3. server.js - Backend API Server

3.1 การตั้งค่า Middleware

```
const express = require('express');
const path = require('path');
const app = express();
const port = process.env.PORT || 5500;

// 1. ตั้งค่า Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// 2. กำหนด CORS Headers
app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content-Type');
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  if (req.method === 'OPTIONS') {
    return res.sendStatus(200);
  }
  next();
});

// 3. กำหนดให้ไฟล์เดิร์ฟปัจจุบันเป็นที่เก็บ Static Files
app.use(express.static(__dirname));
```

🔧 Middleware แต่ละตัว

Middleware	ฟังก์ชัน
express.json()	แปลง JSON body ใน Request ให้เป็น JavaScript Object

Middleware	ฟังก์ชัน
<code>express.urlencoded({ extended: true })</code>	แปลง Form Data (application/x-www-form-urlencoded)
CORS Middleware	อนุญาตให้เรียก API จากโดเมนอื่นได้ (Cross-Origin)
<code>express.static(__dirname)</code>	ให้บริการไฟล์ Static (HTML, CSS, JS, รูปภาพ)

⚠ CORS Preflight Request:

เมื่อ Browser ส่ง Request แบบ POST/PUT/DELETE ที่มี Content-Type ไม่ใช่แบบธรรมดา Browser จะส่ง OPTIONS Request ก่อนเพื่อขออนุญาตจาก Server

3.2 In-Memory Database

```
const likesDatabase = {
  '1': 87,
  '2': 45,
  '3': 120
};

const messagesDatabase = [];
const userCache = {};
const OIL_CACHE_TTL = 5 * 60 * 1000; // 5 นาที
let oilCache = null;
let oilCacheTime = 0;
```

ตัวแปร	คำอธิบาย	ชนิดข้อมูล
likesDatabase	เก็บจำนวนไลค์ของแต่ละ User	Object (Key: userId, Value: number)
messagesDatabase	เก็บข้อความทั้งหมด	Array
userCache	เก็บข้อมูล User ที่ดึงมาจาก JSONPlaceholder	Object (Key: userId, Value: User Data)
OIL_CACHE_TTL	Time-To-Live ของ Cache ราคาน้ำมัน	number (มิลลิวินาที) = 300,000 ms
oilCache	เก็บข้อมูลราคาน้ำมันที่ดึงมาแล้ว	Object หรือ null
oilCacheTime	เวลาที่ดึงข้อมูลราคาน้ำมันครั้งล่าสุด	number (timestamp)

⚠️ ข้อจำกัดของ In-Memory Database:

- ข้อมูลหายเมื่อ Restart Server
- ไม่สามารถแชร์ข้อมูลระหว่าง Server Instance หลายตัวได้
- ไม่มีระบบ Backup และ Recovery

3.3 ฟังก์ชัน Parse ราคาน้ำมัน

```
function parseBangchakOilPrice(html) {
  const oilTypes = [
    'ดีเซล B7', 'แก๊สโซฮอล์ 95', 'เบนซิน 95',
```

```

        'แก๊สโซฮอล์ 91', 'E85', 'ดีเซล B20',
        'ดีเซล พรีเมียม', 'แก๊สโซฮอล์ E20'
    ];

    // ดึงวันที่จาก HTML
    let date = new Date().toLocaleDateString('th-TH');
    const datePatterns = [
        /วันที่\s*(\d{1,2}\s*\d{1,2}\s*\d{4})/,
        /วันที่\s*(\d{1,2}-\d{1,2}-\d{4})/,
        /(\d{1,2}\s*\d{1,2}\s*\d{4})/,
        /(\d{1,2}-\d{1,2}-\d{4})/
    ];

    for (const pattern of datePatterns) {
        const match = html.match(pattern);
        if (match) { date = match[1]; break; }
    }

    // ดึงข้อมูลราคาน้ำมันจากตาราง
    const oilData = [];
    const tableRegex = />([\s\S]*?)<\/table>/i;
    const tableMatch = html.match(tableRegex);
    // ... parsing logic ...

    return {
        date: date,
        prices: oilData,
        note: 'ราคาขายปลีก กทม. ที่ยังไม่รวมภาษีบำรุงท้องถิ่น กทม.'
    };
}

```

ขั้นตอนการทำงาน

1. กำหนดชนิดน้ำมัน - 8 ชนิดตามเว็บบางจาก
2. ค้นหาวันที่ - ใช้ Regular Expression หลากรูปแบบ
3. ดึงข้อมูลราคา - แยกจากโครงสร้าง Table ใน HTML
4. **Fallback** - ถ้าแยกข้อมูลไม่ได้ ให้ใช้ข้อมูลตัวอย่าง

✓ Regular Expression ที่ใช้:

- `/วันที่\s*(\d{1,2}\s*\d{1,2}\s*\d{4})/` - หาวันที่
- `/]*>([\s\S]*?)<\/table>/i` - ดึง Table ทั้งหมด
- `/]*>([\s\S]*?)<\/tr>/gi` - ดึงแต่ละแถว
- `/(\d{2,3}\.\d{2})/g` - ดึงราคา (เช่น 32.50, 37.50)

3.4 API Endpoints ทั้งหมด

Method	Endpoint	คำอธิบาย	Cache
GET	/api/oil-price	ดึงข้อมูลราคา น้ำมันบางจาก (JSON)	✓ 5 นาที
GET	/api/oil-raw	ดึงข้อมูลราคา น้ำมันบางจาก (HTML ดิบ)	✗
GET	/api/users/:id	ดึงข้อมูลผู้ใช้จาก JSONPlaceholder	✓ ตลอด ไป
GET	/api/users	ดึงข้อมูลผู้ใช้ ทั้งหมด	✗

Method	Endpoint	คำอธิบาย	Cache
GET	/api/posts?limit=10	ดึงข้อมูลโพสต์ จาก JSONPlaceholder	✗
GET	/api/likes/:userId	ดึงยอดไลค์ของผู้ ใช้	✗
POST	/api/likes/:userId	อัปเดตยอดไลค์ (like/unlike)	✗
POST	/api/messages	บันทึกข้อความ	✗
GET	/api/messages	ดึงข้อความ ทั้งหมด	✗
GET	/api/messages/user/:userId	ดึงข้อความตามผู้ ใช้	✗
GET	/api/all	ดึงข้อมูลทั้งหมด (รวมข้อมูลจาก หลายแหล่ง)	✓ บาง ส่วน

ตัวอย่างการเรียกใช้ API

GET /api/users/1

```
// Response
{
  "success": true,
  "data": {
    "id": 1,
```

```
"name": "Leanne Graham",
"username": "Bret",
"email": "Sincere@april.biz"
},
"source": "JSONPlaceholder",
"timestamp": "2026-06-27T10:30:00.000Z"
}
```

POST /api/likes/1

```
// Request Body
{ "action": "like" }

// Response
{
  "success": true,
  "userId": "1",
  "likes": 88,
  "action": "like",
  "timestamp": "2026-06-27T10:30:00.000Z"
}
```

4. HTML Frontend - การเชื่อมต่อกับ Backend

4.1 การโหลดข้อมูลผู้ใช้

```
async function loadUserData() {
  try {
    userNameDisplay.textContent = 'กำลังโหลด...';
    const response = await fetch('https://jsonplaceholder.typicode.com/us
```

```

    const user = await response.json();
    userNameDisplay.textContent = user.name;
    isUserLoaded = true;

    const likesResponse = await fetch('/api/likes/1');
    const likesData = await likesResponse.json();
    likeCount = likesData.likes;
    updateLikeDisplay();
  } catch (error) {
    userNameDisplay.textContent = 'ไม่พบข้อมูล';
  }
}

```

4.2 การกด Like

```

likeBtn.addEventListener('click', async function() {
  if (!isUserLoaded) return;

  isLiked = !isLiked;
  const action = isLiked ? 'like' : 'unlike';

  const response = await fetch(`/api/likes/1`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ action: action })
  });

  const data = await response.json();
  likeCount = data.likes;
  updateLikeDisplay();
});

```

4.3 การส่งข้อความ

```

async function sendMessage(message) {
  const response = await fetch('/api/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },

```

```
        body: JSON.stringify({ userId: '1', message: message })
    });
    const result = await response.json();
    if (result.success) {
        console.log('บันทึกข้อความสำเร็จ:', result.data);
    }
}
```

5. การทำงานแบบ End-to-End

สถานการณ์ที่ 1: ผู้ใช้โหลดหน้าเว็บ

1. Browser: GET /index.html

Server: ส่งไฟล์ HTML กลับ

2. Browser: โหลด CSS และ JavaScript

Server: ส่งไฟล์ Static

3. JavaScript: fetch('/api/users/1')

Server: GET /api/users/1

Server: ตรวจสอบ Cache → ไม่มี

Server: fetch('https://jsonplaceholder.typicode.com/users/1')

JSONPlaceholder: ส่งข้อมูล User

Server: เก็บ Cache, ส่ง Response

Browser: แสดงชื่อผู้ใช้

4. JavaScript: fetch('/api/likes/1')

Server: GET /api/likes/1

Server: likesDatabase['1'] → 87

Browser: แสดงยอดไลค์ 87

✦ สถานการณ์ที่ 2: ผู้ใช้กด Like

1. **User:** คลิกปุ่ม Like

2. **JavaScript:** `fetch('/api/likes/1', { method: 'POST', body: JSON.stringify({ action: 'like' }) })`

3. **Server:** POST /api/likes/1

- อ่าน { action: 'like' } จาก req.body
- likesDatabase['1'] = 88
- ส่ง Response { likes: 88 }

4. **Browser:** อัปเดตยอดไลค์เป็น 88 และเปลี่ยนปุ่มเป็น "เลิกไลค์"

✦ สถานการณ์ที่ 3: ดึงราคาน้ำมัน (มี Cache)

1. **JavaScript:** `fetch('/api/oil-price')`

2. **Server:** GET /api/oil-price

- ตรวจสอบ Cache: oilCache มีข้อมูล, ยังไม่หมดอายุ
- ส่ง Response จาก Cache ทันที (ไม่ต้องไปดึงเว็บจาก)

3. **Browser:** แสดงราคาน้ำมัน

- source: 'Cache'
- timestamp: เวลาที่ดึงข้อมูลครั้งล่าสุด

6. การดีบั๊กและการทดสอบ

การทดสอบด้วย cURL

```
# ทดสอบ GET /api/users/1
curl http://localhost:5500/api/users/1

# ทดสอบ POST /api/likes/1
curl -X POST http://localhost:5500/api/likes/1 \
  -H "Content-Type: application/json" \
  -d '{"action": "like"}'

# ทดสอบ POST /api/messages
curl -X POST http://localhost:5500/api/messages \
  -H "Content-Type: application/json" \
  -d '{"userId": "1", "message": "Hello DevPool"}'

# ทดสอบ GET /api/all
curl http://localhost:5500/api/all
```

การตรวจสอบด้วย Chrome DevTools

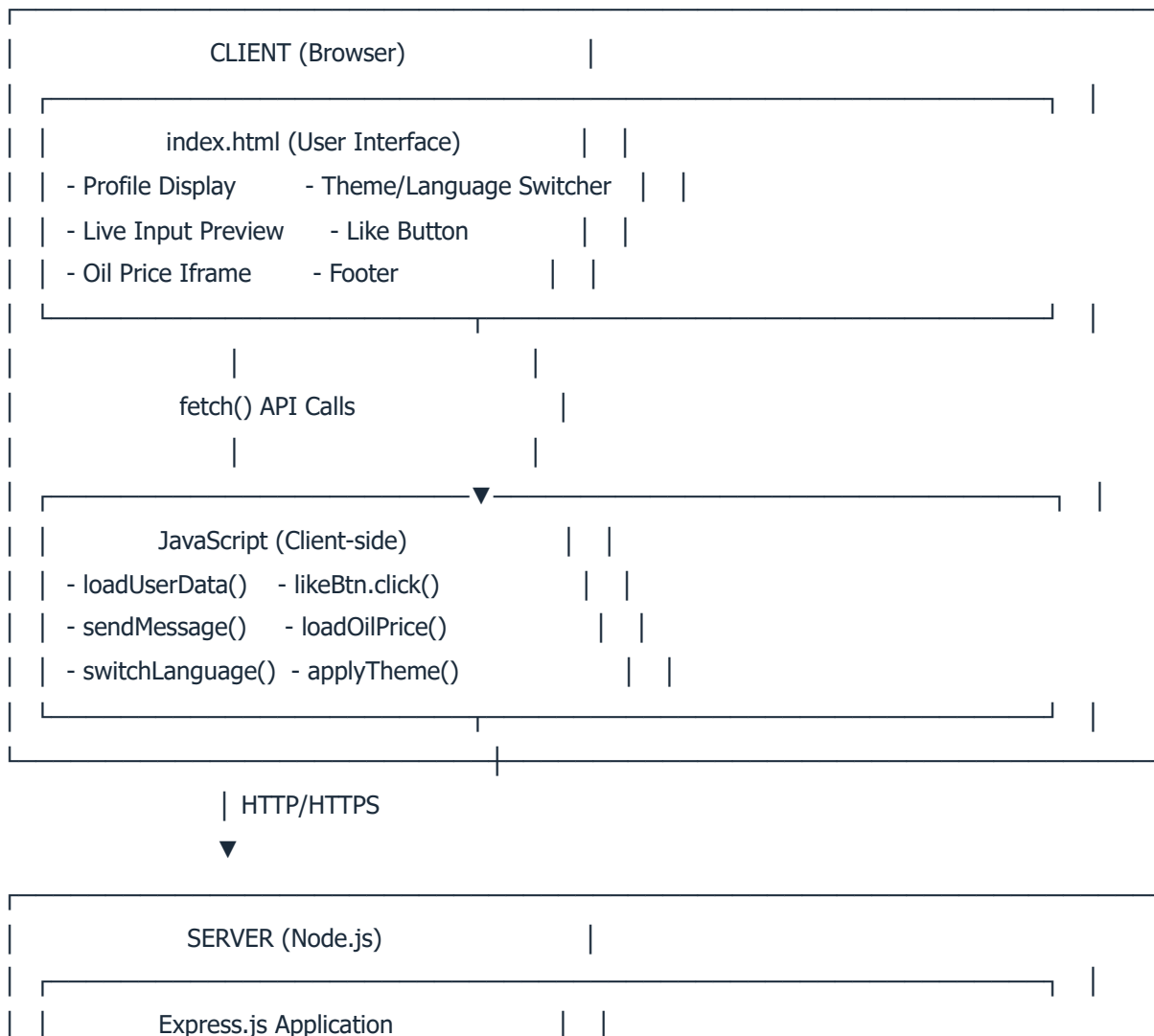
- **Network Tab** - ดู Request/Response ทั้งหมด, Status Code, Headers, Response Body
- **Console Tab** - ดู Log และ Error Messages
- **Application Tab** - ตรวจสอบ Local Storage, Session Storage

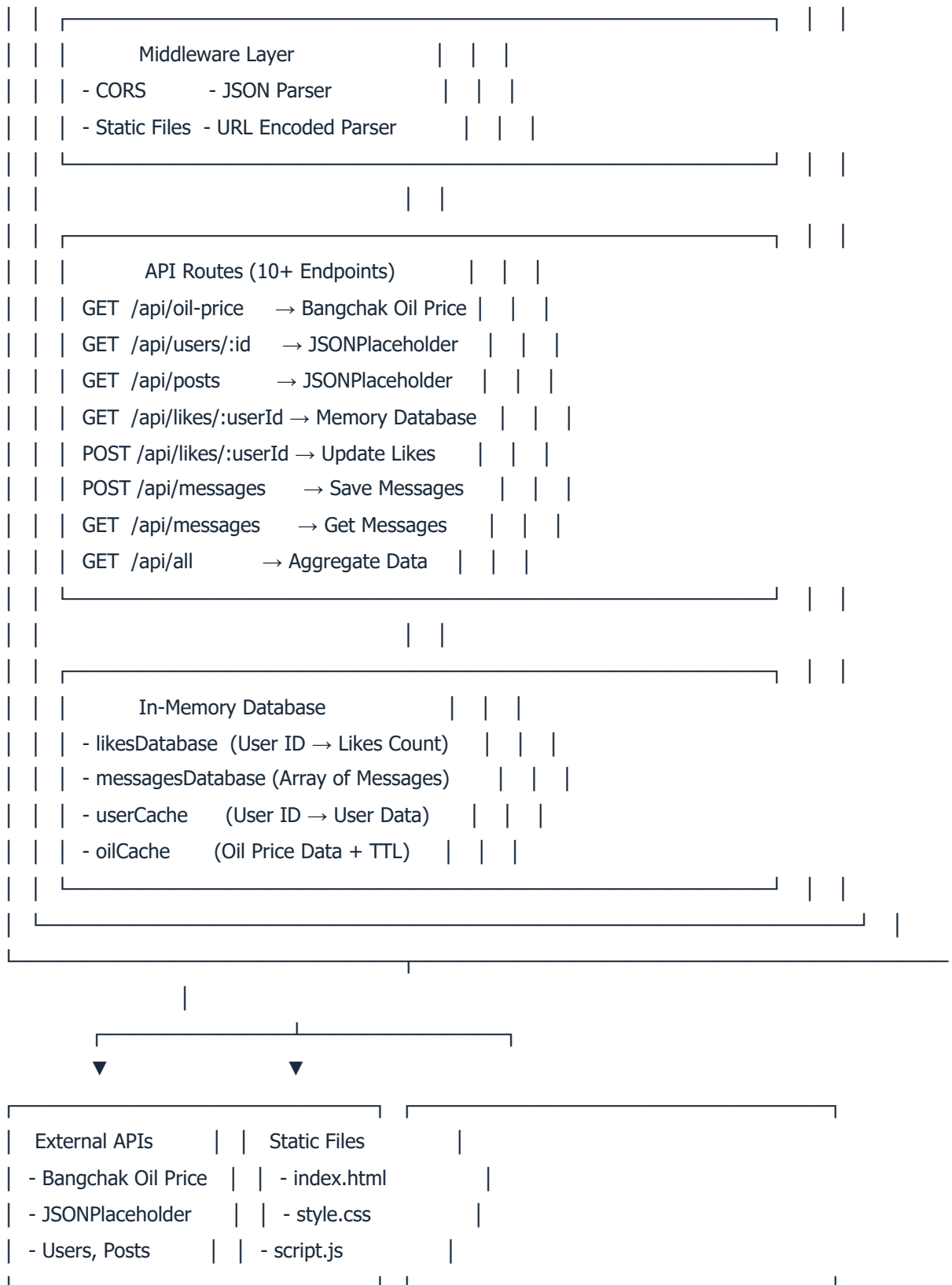
การตรวจสอบ Log ใน Server

```
// ใน server.js
console.log('🚀 Server is running on http://localhost:5500');
console.log(`🔥 GET /api/oil-price - ${new Date().toISOString()}`);

// ใน API Handler
console.log(`[${new Date().toISOString()}] GET /api/users/${userId}`);
```

7. สถาปัตยกรรมระบบ





8. สรุปและข้อควรระวัง

✅ ข้อดี

- **Separation of Concerns** - แยก Frontend และ Backend ชัดเจน
- **Caching** - ลดการเรียก API ภายนอก เพิ่มความเร็ว
- **Error Handling** - มี Fallback เมื่อ API ภายนอกล้มเหลว
- **CORS Ready** - รองรับการเรียกจากโดเมนอื่น
- **Responsive Design** - รองรับทุกอุปกรณ์ (Desktop, Tablet, Mobile)
- **Multi-language** - รองรับภาษาไทยและอังกฤษ
- **Theme Switching** - รองรับ Light, Dark, Auto Mode

⚠️ ข้อควรระวัง

- **In-Memory Database** - ข้อมูลหายเมื่อ Restart Server
- **Cache TTL** - ข้อมูลราคาน้ำมันอาจไม่ทันสมัย (5 นาที)
- **No Authentication** - ไม่มีระบบ Login/Authorization
- **Single Point of Failure** - ถ้า Server Crash ข้อมูลทั้งหมดหาย
- **Memory Usage** - messagesDatabase จะเก็บข้อมูลเรื่อยๆ (จำกัดที่ 100)

🚀 แนวทางการปรับปรุง

- ใช้ Database จริง (MongoDB, PostgreSQL, SQLite)
- เพิ่ม Authentication (JWT, Session)
- ใช้ Redis แทน In-Memory Cache
- เพิ่ม Rate Limiting ป้องกัน DDoS

- ใช้ Docker สำหรับ Deployment
 - เพิ่ม WebSocket สำหรับ Real-time Communication
 - เพิ่ม Unit Test และ Integration Test
 - ใช้ Environment Variables สำหรับ Configuration
-

DevPool 2026 · Full-Stack Web Application

Built with ❤️ using HTML, CSS, JavaScript, Node.js, Express

เอกสารนี้สร้างเมื่อ: 29 มิถุนายน 2569 เวลา 00:21